Guided Research Report: Userspace Page Faults for Hardware-Accelerated Data Caching

Anton Ge
Advisors: Ilya Meignan–Masson, David Schall Supervisor: Pramod Bhatotia
System Research Group, Technical University of Munich

27.10.2026

1 Motivation

Database systems such as key-value stores and graph engines process datasets larger than memory, so they need a data cache that moves pages between storage and memory. *OS paging* (`mmap`) is simple: data is accessed through pointers, the memory management unit (MMU) translates addresses, and resident accesses run at native speed. However, the kernel owns the replacement policy without knowing the access patterns, and its fault path is slow and scales poorly [3, 9]. *Explicit buffer managers* [7, 6] give the application control over the policy, but they check residency in software on every access and require all data structures to be built around explicit page fixing.

Userspace paging promises both: the hardware detects missing pages and an application-defined handler resolves them. The existing Linux mechanism, `userfaultfd` [8], traps every fault in the kernel, forwards it to a handler thread through a file descriptor, and re-enters the kernel to install the page. This communication accounts for 20–50% of the fault latency, which has kept userspace paging out of production systems [10]. This project contributes to Ricochet, a userspace paging architecture co-designed with the processor's translation hardware. Its core is the *userspace page fault* (UPF), a hardware primitive that delivers page faults directly to a handler in the faulting thread, without entering the kernel.

2 Userspace Page Faults

UPF builds on *user interrupts* (UINTR) [4, 5], an Intel Sapphire Rapids feature that delivers interrupts to a userspace handler without a kernel transition. UINTR cannot serve page faults as is: it delivers every interrupt of a thread, whereas UPF must fire only for faults in cache regions; it is triggered by an instruction (`senduipi`), whereas UPF must be raised by the MMU on the faulting thread; and it conveys only the sender's identity, whereas a fault handler needs the faulting address. Figure 1 shows how UPF addresses each point.

Fault selection in the page table. Ricochet reuses a software-available bit of the page-table entry (PTE) as the *UPF bit*. During a page-table walk, the MMU tests it together with the present bit: a non-present page with the UPF bit set raises a UPF (step 2), and every other fault takes the ordinary kernel path. Unlike range registers in the MMU, this check needs no extra hardware state or memory accesses and scales to any number of regions.

Synchronous delivery. User interrupts are delivered between instructions, but a page fault must stop the faulting instruction. UPF routes the MMU's exception signal into the user-interrupt delivery logic of the core's local APIC (step 3). The core halts the pipeline at the faulting instruction, pushes a user-level exception frame onto the thread's stack, masks further user-level deliveries on that core, and jumps to the handler that the thread registered.

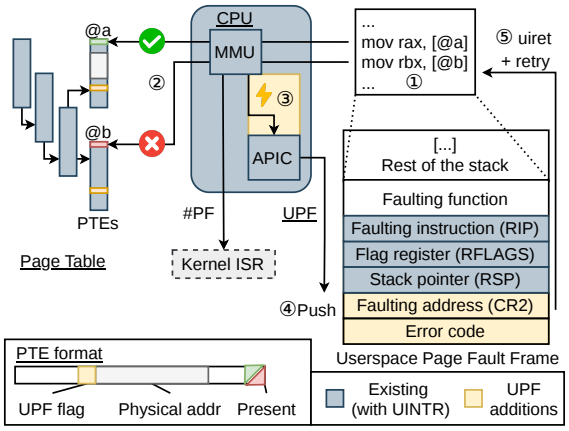


Figure 1: Userspace page fault handling. The hardware additions are the UPF bit in the page-table entry, the hand-off from the MMU to the local APIC, and two extra fields in the exception frame.

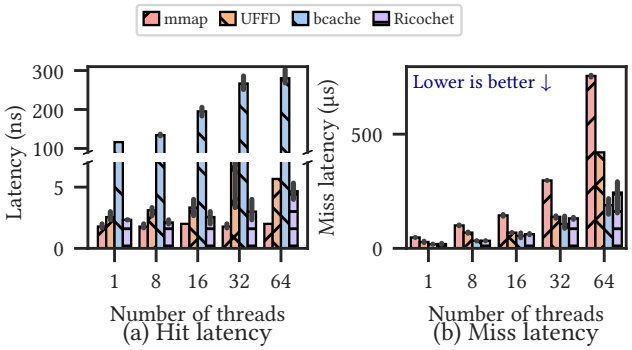


Figure 2: Fault handling without replacement: hit latency (a) and miss latency (b) from 1 to 64 threads.

Extended exception frame. A UINTR frame holds the interrupted instruction pointer, flags, stack pointer, and the sender's vector. UPF adds the faulting virtual address and the error code that encodes the access type (step 4). The processor already holds both values when it raises the exception, so the frame costs no extra memory accesses.

Handler return. After resolving the fault, the handler executes `uiret`, which unwinds the frame, re-enables user-level delivery, and returns to the faulting instruction (step 5), which then re-executes as after a kernel-resolved fault. A userspace handler cannot edit page tables, so a thin kernel *monitor* installs pages and unmaps evicted ones in batches, while a userspace *library* runs the rest of the resolution and lets applications customize the replacement policy and I/O. Hits involve neither: they are plain memory accesses translated by the MMU.

3 Implementation and Evaluation

We model UPF in the gem5 full-system simulator [2], extending the UINTR model of xUI [1] (an out-of-order x86-64 core configured after Sapphire Rapids, Linux 6.0 guest). The page-table walker tests the UPF bit and, on a matching fault, sends a delivery request to the local APIC instead of trapping to the kernel; the APIC builds the extended frame and vectors the core to the handler. The hardware model adds 436 lines to xUI.

We measure the cost of fault handling without replacement with a microbenchmark: worker threads issue random single-page reads by pointer dereference over a 16 GiB file-backed region, and every access is timed, separating hits from misses, from 1 to 64 threads. We compare Ricochet with `mmap`, `userfaultfd` with a pool of handler threads, and a block cache with software guards following vmcache [6].

Figure 2a shows that Ricochet keeps the native hit path: resident accesses take 2–5 ns at every thread count, like `mmap`, whereas the block cache's per-access check costs 116–280 ns. On misses (Figure 2b), Ricochet resolves a single-threaded fault in 19.0 µs, comparable to the block cache (18.3 µs) and 1.5× and 2.5× faster than `userfaultfd` and `mmap`. It stays comparable to the block cache at every thread count, and at 64 threads it resolves a miss 1.7× faster than `userfaultfd` and 3.1× faster than `mmap`.

4 Conclusion

UPF moves page-fault notification out of the kernel and into the hardware path that already detects missing pages. With a PTE bit for selection, synchronous delivery through the user-interrupt engine, and two extra frame fields, applications handle their own page faults while keeping pointer-based access: hits run at native speed and misses cost as much as in a custom buffer manager.

References

[1] Berk Aydogmus, Linsong Guo, Danial Zuberi, Tal Garfinkel, Dean Tullsen, Amy Ousterhout, and Kazem Taram. Extended user interrupts (xui): Fast and flexible notification without polling. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*, ASPLOS '25, page 373–389, New York, NY, USA, 2025. Association for Computing Machinery.

[2] Nathan Binkert, Bradford Beckmann, Gabriel Black, Steven K. Reinhardt, Ali Saidi, Arkaprava Basu, Joel Hestness, Derek R. Hower, Tushar Krishna, Somayeh Sardashti, Rathijit Sen, Korey Sewell, Muhammad Shoaib, Nilay Vaish, Mark D. Hill, and David A. Wood. The gem5 simulator. *SIGARCH Comput. Archit. News*, 39(2):1–7, August 2011.

[3] Andrew Crotty, Viktor Leis, and Andrew Pavlo. Are You Sure You Want to Use MMAP in Your Database Management System? In *Proceedings of the 2022 Conference on Innovative Data Systems Research*, 2022.

[4] Intel. Intel 64 and IA-32 Architectures Software Developer's Manual. `https://www.intel.com/content/www/us/en/developer/articles/technical/intel-sdm.html`. Accessed: 2026-07-16.

[5] Yves Kone, Louis Duval, Renaud Lachaize, Pascal Felber, Daniel Hagimont, and Alain Tchana. Understanding intel user interrupts. *Proc. ACM Meas. Anal. Comput. Syst.*, 9(2), June 2025.

[6] Viktor Leis, Adnan Alhomssi, Tobias Ziegler, Yannick Loeck, and Christian Dietrich. Virtual-Memory Assisted Buffer Management. *Proceedings of the ACM on Management of Data*, 1(1), 2023.

[7] Viktor Leis, Michael Haubenschild, Alfons Kemper, and Thomas Neumann. Leanstore: In-memory data management beyond main memory. In *2018 IEEE 34th International Conference on Data Engineering (ICDE)*, pages 185–196. IEEE, 2018.

[8] Linux Kernel Developers. Userfaultfd: The Linux Kernel Documentation. `https://www.kernel.org/doc/html/next/admin-guide/mm/userfaultfd.html`. Accessed: 2026-07-16.

[9] Anastasios Papagiannis, Giorgos Xanthakis, Giorgos Saloustros, Manolis Marazakis, and Angelos Bilas. Optimizing Memory-mapped I/O for Fast Storage Devices. USENIX Association, 2020.

[10] Ivy B. Peng, Maya B. Gokhale, Karim Youssef, Keita Iwabuchi, and Roger Pearce. Enabling Scalable and Extensible Memory-Mapped Datastores in Userspace. *IEEE Transactions on Parallel and Distributed Systems*, 33(4), 2022.